

# Generative Models

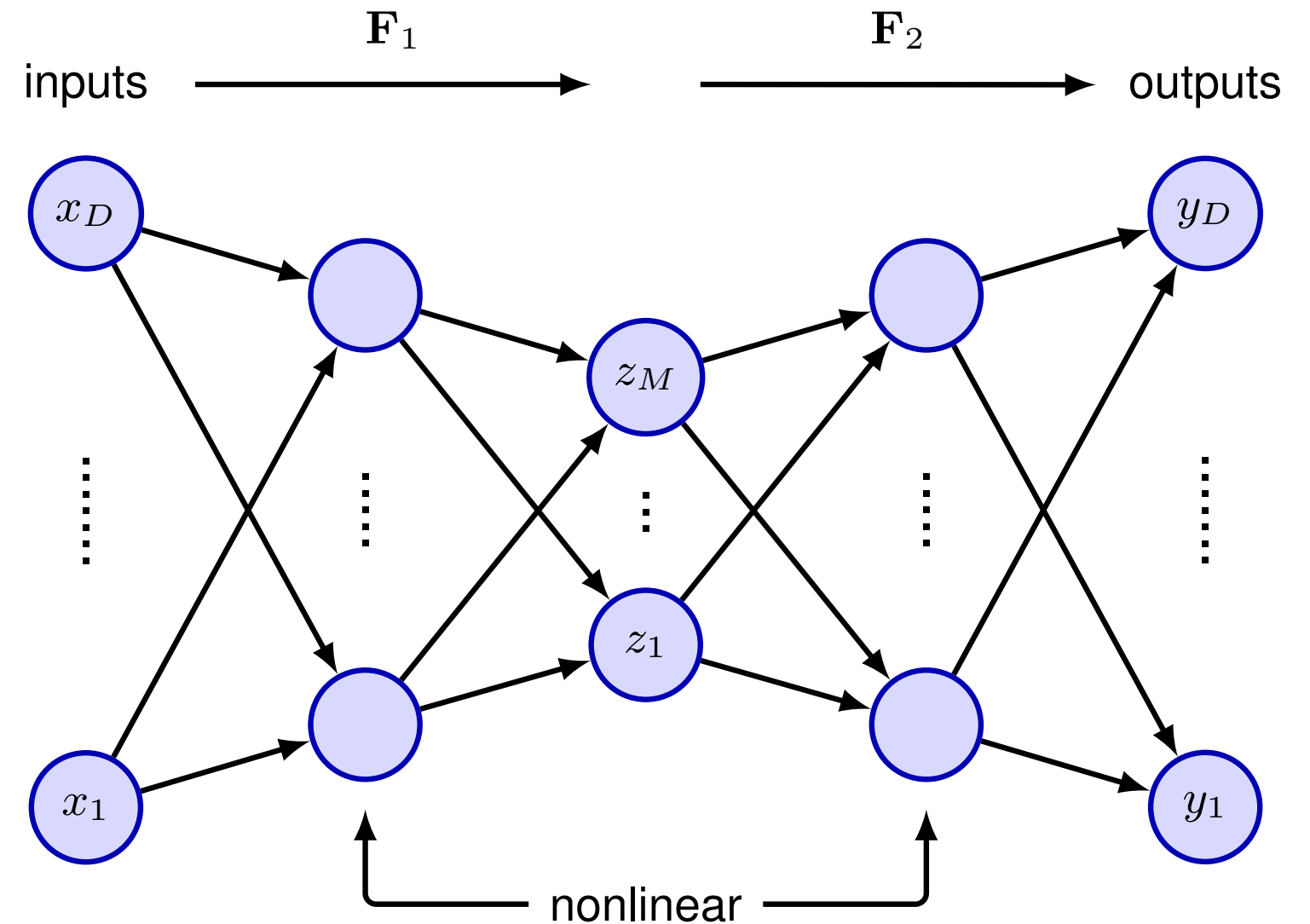
AI506: Advanced Machine Learning

Spring 2026

Lukas Galke

# Brief recap: Autoencoders come in many variants

- Linear/nonlinear AEs
- Shallow/deep AEs
- Under-/overcomplete AEs
- Regularized AEs
  - Denoising AEs
  - Sparse AEs
  - Variational AEs
- ...



# What are generative models

The goal is to learn the underlying data-generating distribution  $p(x)$ , such that you can sample from it.

# Energy-based Models and why Generative Modeling is hard!

Idea: Let's just learn an arbitrary model to predict probabilities  $p(x)$ .

Energy based models:  $p(x) \approx \frac{e^{-\beta f_{\theta}(x)}}{Z(\theta)}$

where  $Z$  is the **partition function**, used for normalizing the output of the energy-based model to form a probability distribution.

You can express  $Z = \int_{x \in \mathcal{X}} e^{-\beta f_{\theta}(x)} dx$

**But integrating over the whole space  $\mathcal{X}$  is usually intractable.**

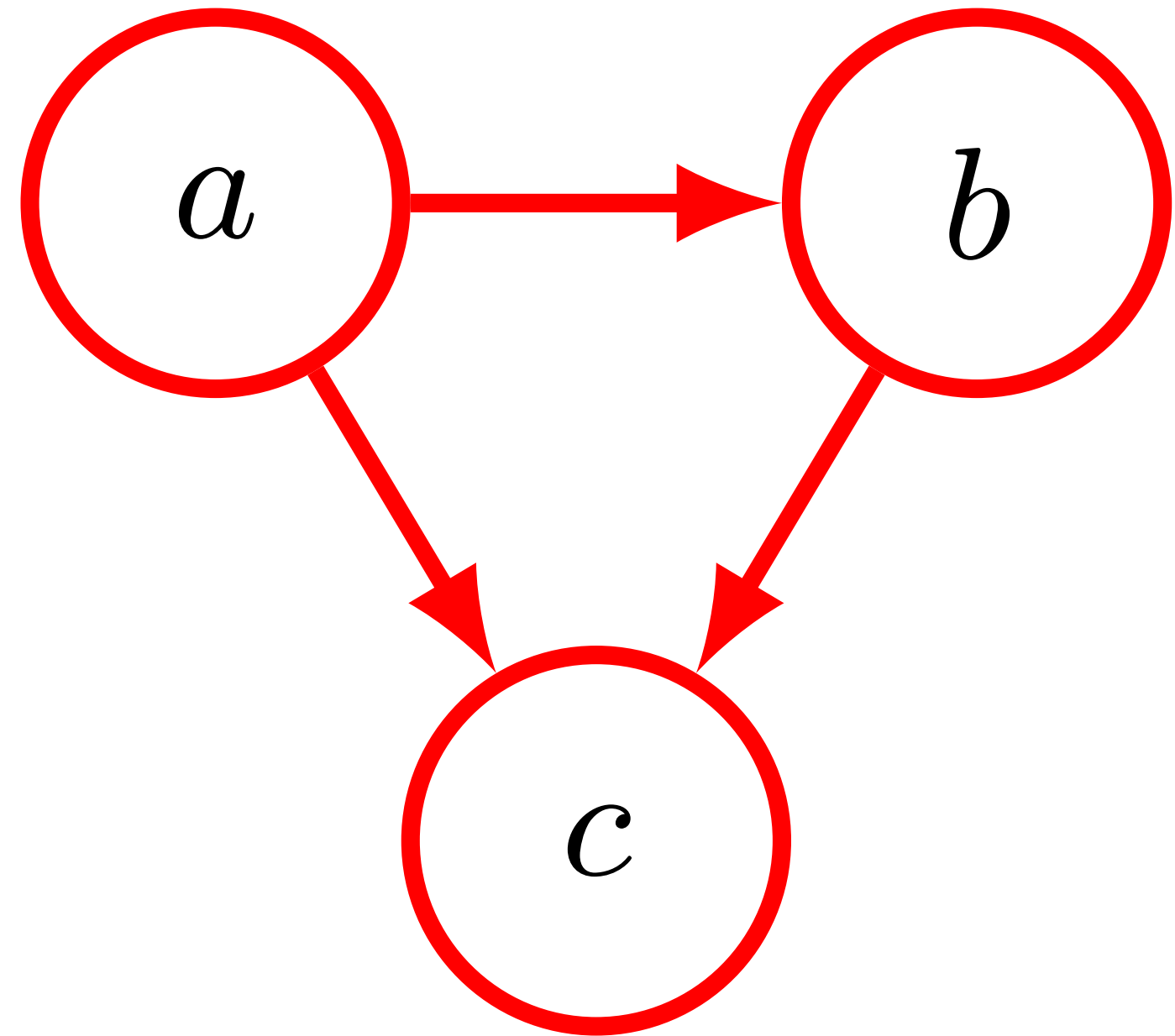
# Keywords

- Energy-based models
- Probabilistic graphical models
- Bayesian Networks vs. Markov Networks
- Neural networks as generative models
- Generative adversarial networks

# Probabilistic Graphical Models

# Directed Graphs

- A set of nodes
- A set of edges, each connecting two nodes.  
In probabilistic graphical models:
- Each node is a random variable
- Each edge is a probabilistic relationship between the random variables





## Factorization

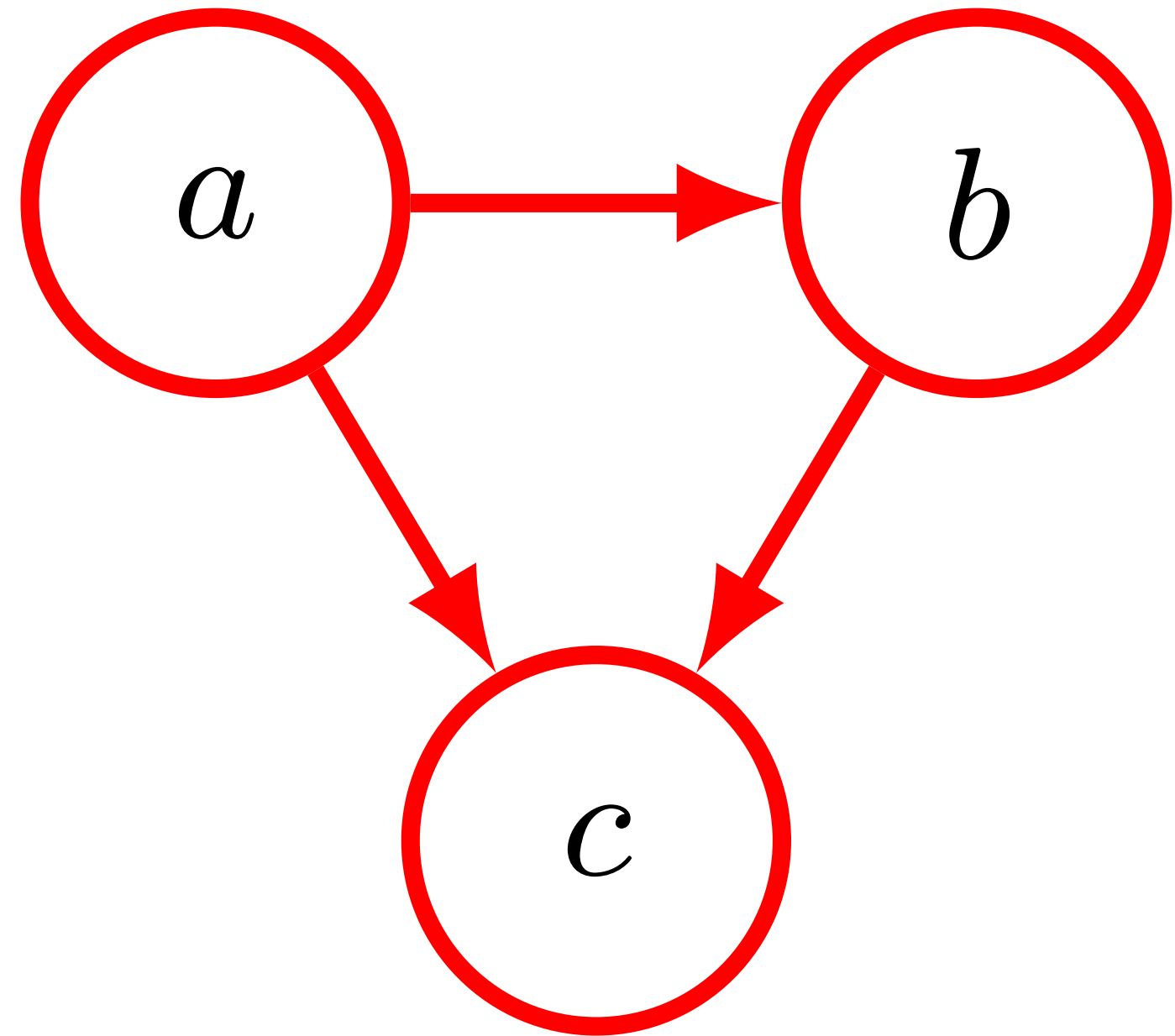
Any joint probability distribution can be factorized.

$$p(a, b, c) = p(c|a, b)p(a, b)$$

and further:

$$p(a, b, c) = p(c|a, b)p(b|a)p(a)$$

**Note** that an arrow  $x \rightarrow y$  reads is interpreted as the conditional probability  $p(y|x)$

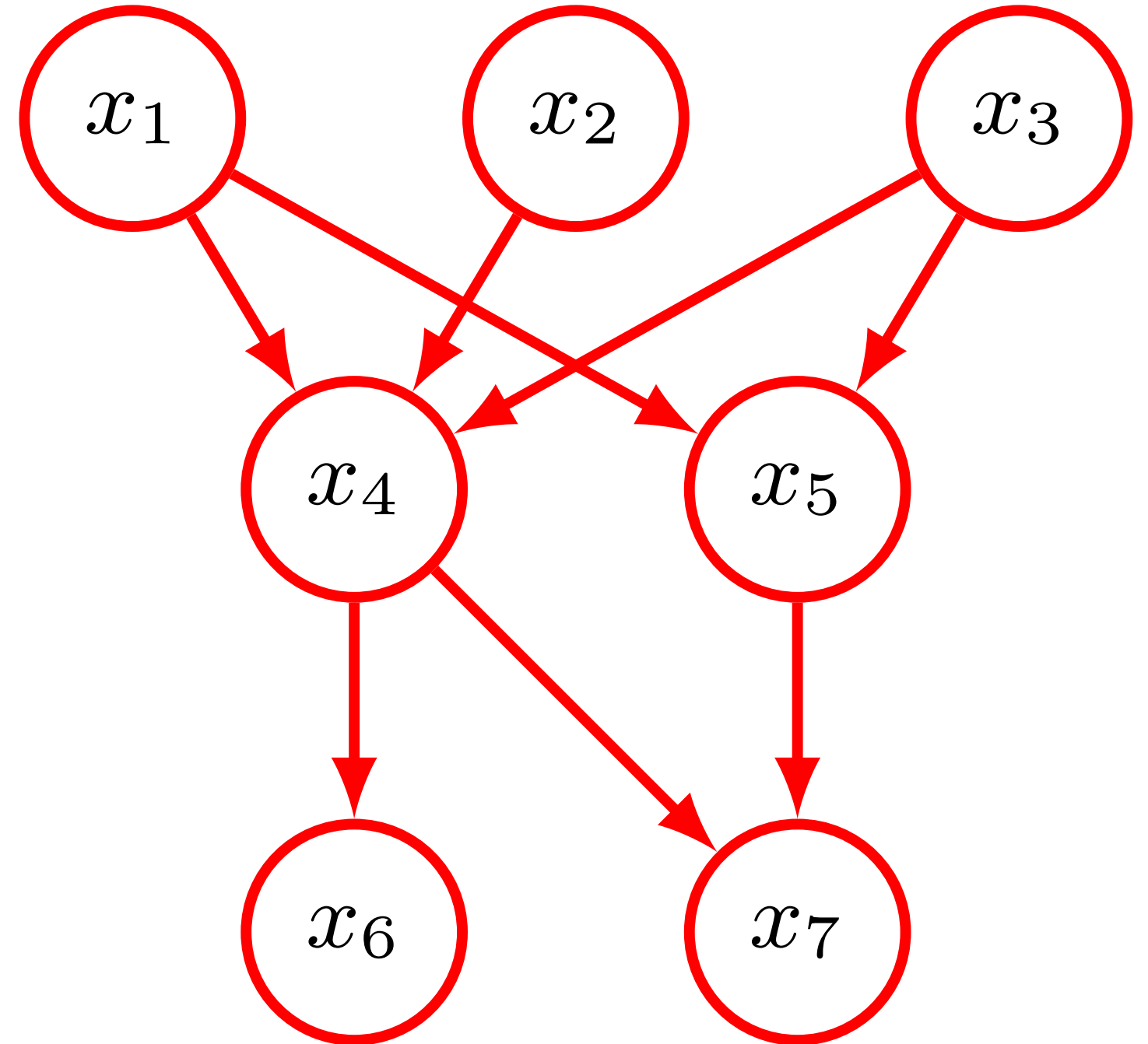




## A more complex example

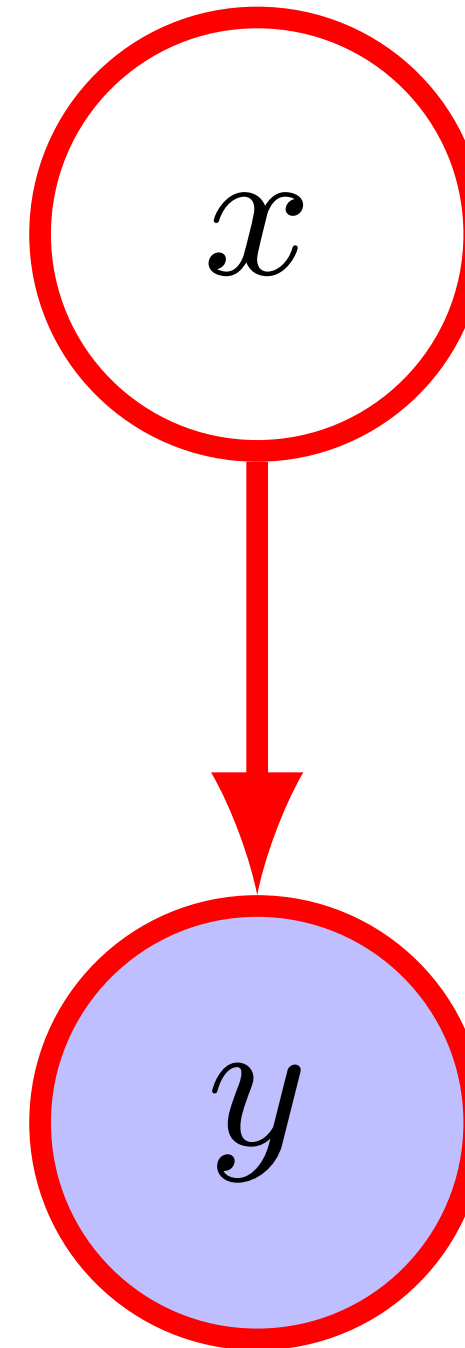
$$\begin{aligned} p(x_1, x_2, x_3) &= \\ p(x_1)p(x_2)p(x_3) \cdot \\ p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3) \cdot \\ p(x_6|x_4) \cdot p(x_7|x_4, x_5) \end{aligned}$$

Nodes can be discrete (e.g., Bernoulli) or continuous (e.g., Gaussian)



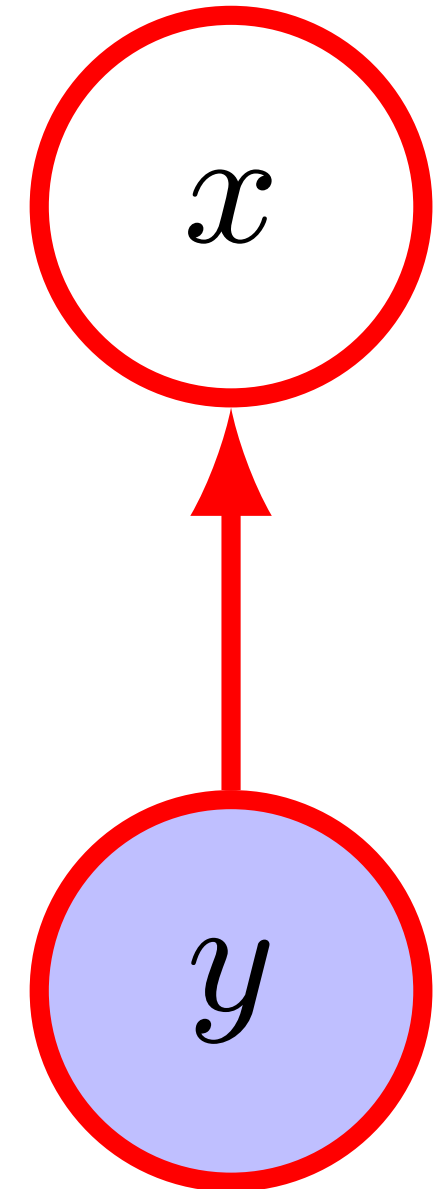
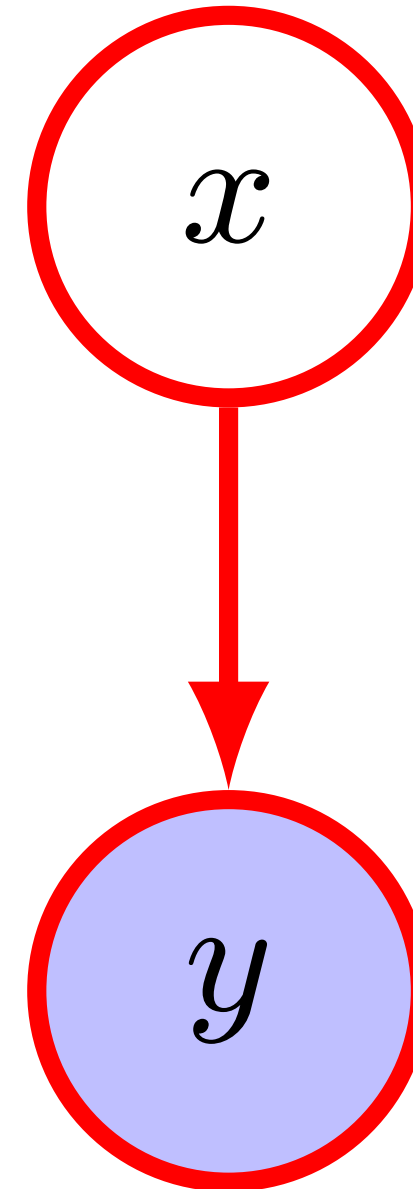
# Observed and latent variables

- Nodes can correspond to **observed** = what we see in the data
- ...or **latent/hidden** variables = underlying factors of variation



# Inference

- Suppose we observe a value of  $y$
- Prior distribution  $p(x)$
- $p(x|y) = \frac{p(y|x)p(x)}{p(y)}$  (Bayes)
- We can update our  $p(x)$  accordingly to what was observed
- Similar calculations can be done for more complex graphs.



# Expectation-Maximization algorithm

1. Initialize parameters with an initial guess
2. Expectation step: Calculate the probability of each data point belonging to each possible hidden variable based on current parameters.
3. Update the parameters to **maximize the log-likelihood of the data**, using probabilities calculated in the E-step.
4. Iterate: Repeat 2. And 3. until parameters stabilize.

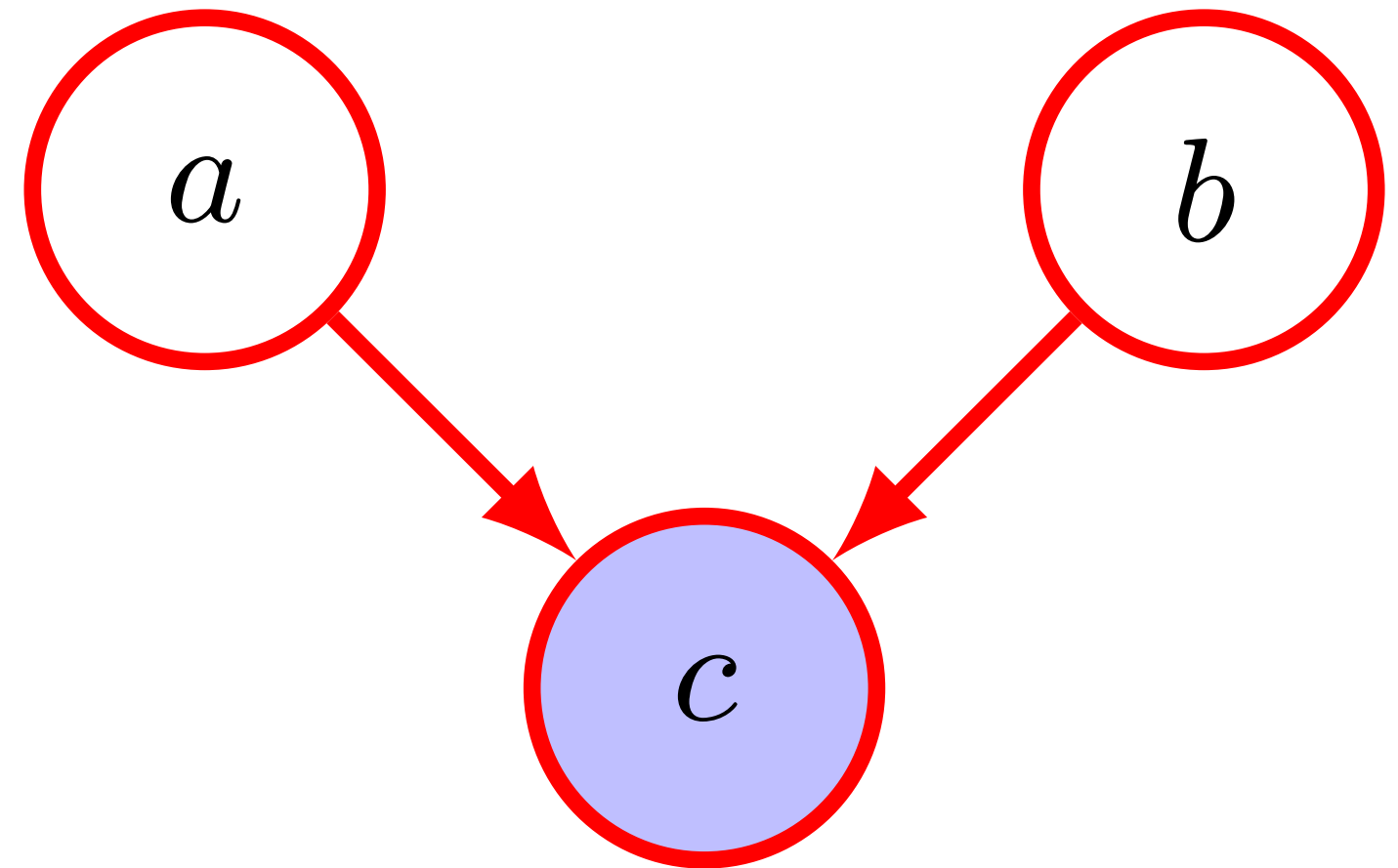
## Key Problem: Explaining away effect

Two independent causes become conditionally dependent given their common effect.

**Example:** Earthquake (a) and Burglary (b) may both cause an alarm (c).

An observed alarm (c) will increase our belief of both earthquake (a) and burglary (b).

But if we then knew it was actually an earthquake (a), we can decrease our belief in a burglary (b).



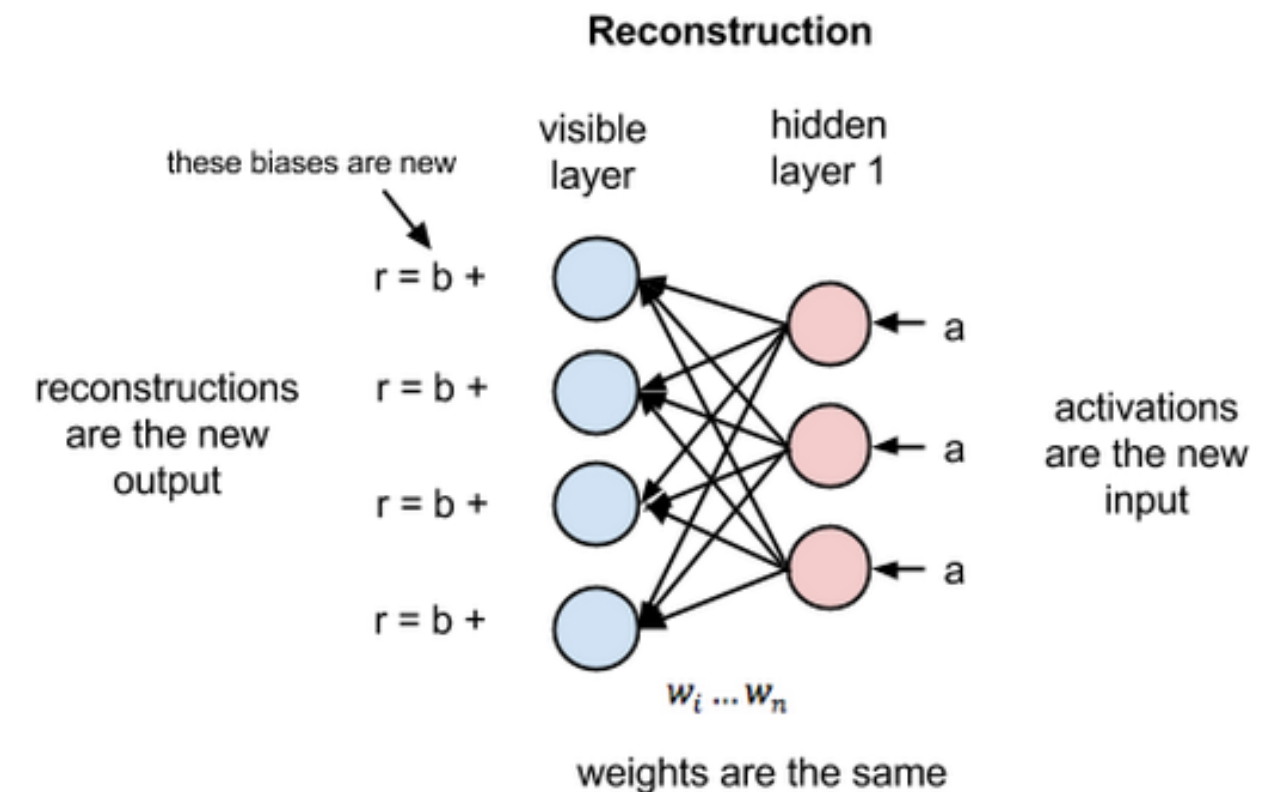
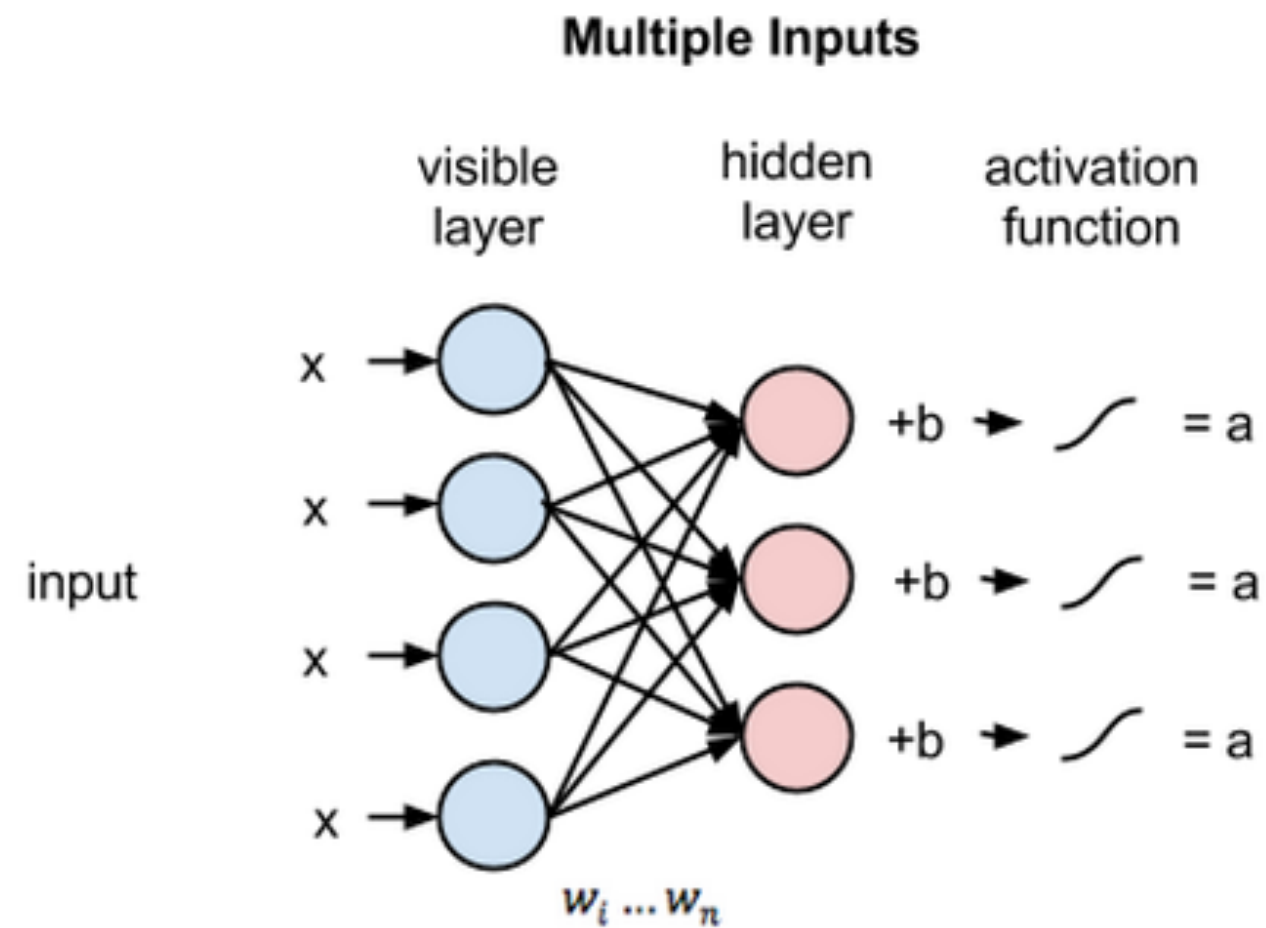
# Major families of probabilistic graphical models

1. **Bayesian networks:** Directed acyclic graph
2. **Markov networks:** Undirected graph
  - Special case: **Restricted Boltzmann Machines** (1 layer of observed, and 1 layer of hidden units).

# Restricted Boltzmann Machines are similar to Autoencoders

Step 1: Inferring latent variables from observed ones

Step 2: Reconstructing observed variables from latent ones





# Why are probabilistic graphical models **generative**?

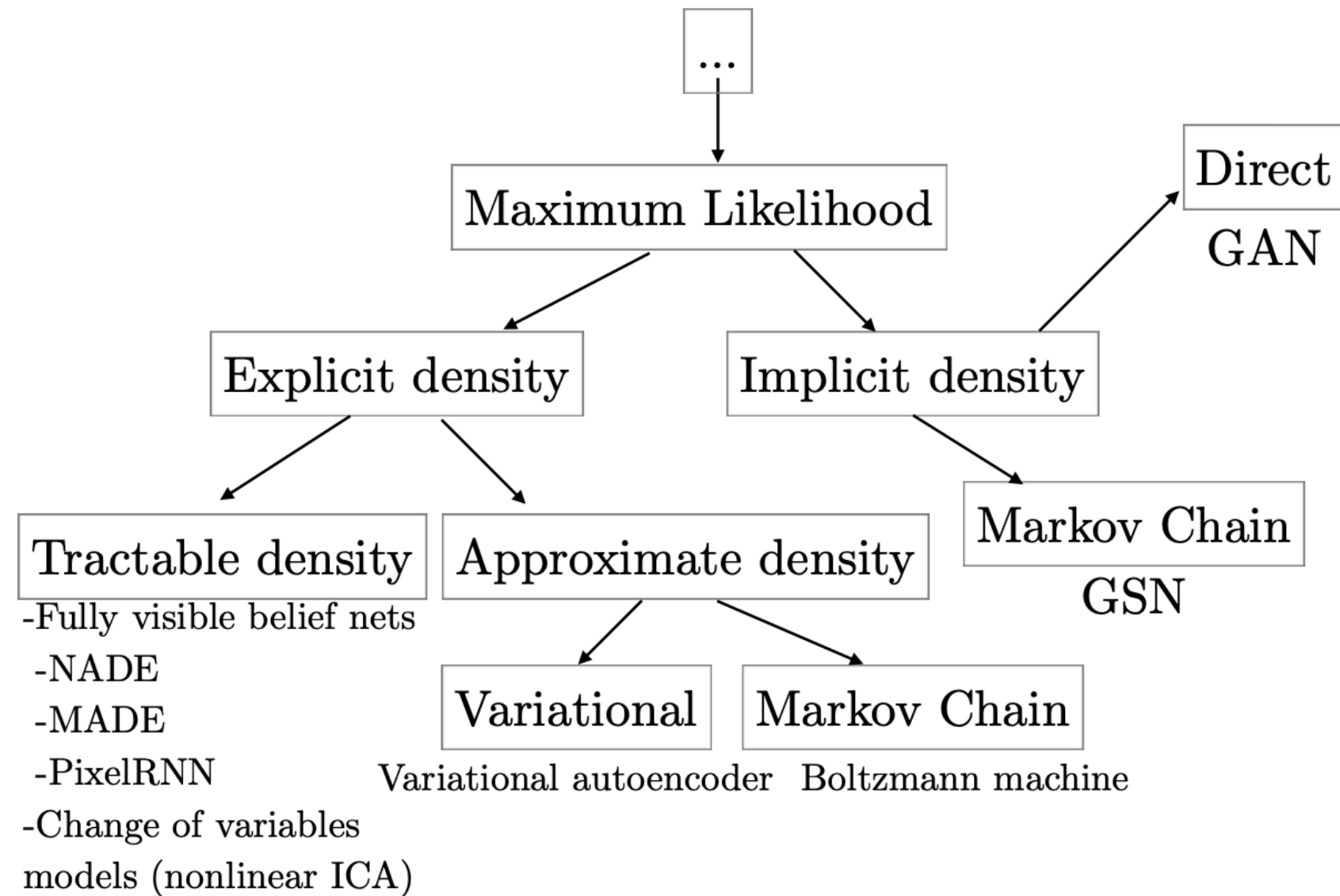
You can sample from them, as you model the full joint probability distribution.



## ***Summary: Probabilistic Graphical Models***

- Probabilistic graphical models are different, but still somewhat related to neural nets (which are deterministic)
- Some concepts are carried over to neural nets (e.g., factorization, having latent variables, the term "hidden units")
- Deep learning has a common history, coming from 90% neural nets, but also 10% probabilistic graphical models
- In deep neural networks models you allow maximum flexibility for your learnable parameters, and do not need to design the graph by hand.

# A Birds-Eye perspective



# Neural Networks as Generative Models

We want to just give the model data and have it figure everything out by itself!

We know already two types of generative models based on deep neural networks. **Question:** which ones?

- Variational Autoencoders
- Decoder-only Transformers

We know already two types of generative models based on deep neural networks:

- Decoder-only Transformers
- Variational Autoencoders

# Decoder-only Transformers are Generative Sequence Models

Goal is given a dataset  $\{x_i\}_{I < N}$ , learn  $p(x)$ .

Idea of sequence models:

$$p(x) = \prod_{I=0}^N p(x_i | x_{<i})$$

Unrolled:  $p(x) = p(x_0) \cdot p(x_1 | x_0) \cdot p(x_2 | x_0, x_1) \cdot \dots$



# Variational Autoencoder Revisited

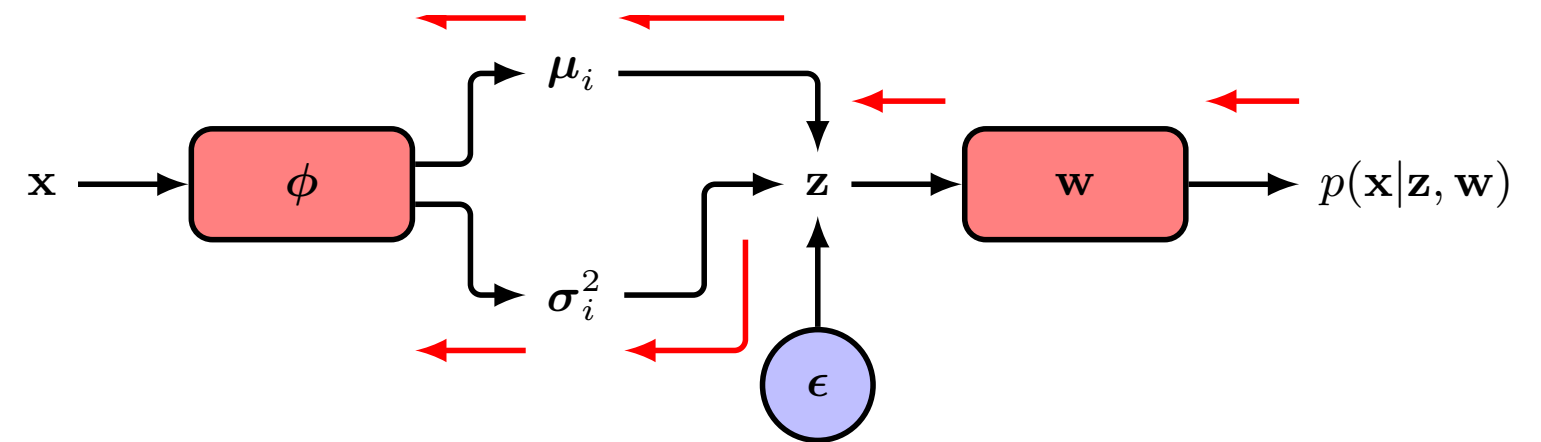
## Reparametrization trick

$$z = \mu + \sigma * \epsilon$$

where  $\mu$  and  $\sigma$  are predicted, but  
 $\epsilon \sim \mathcal{N}(0, 1)$

Question 1: Why do you need the  
reparametrization trick?

Question 2: How can you sample  
from the model?





# Maximize Variational Lower-Bound

Consider a VAE with a probabilistic encoder  $q_\phi(z|x)$  and decoder  $p_\theta(x|z)$ .

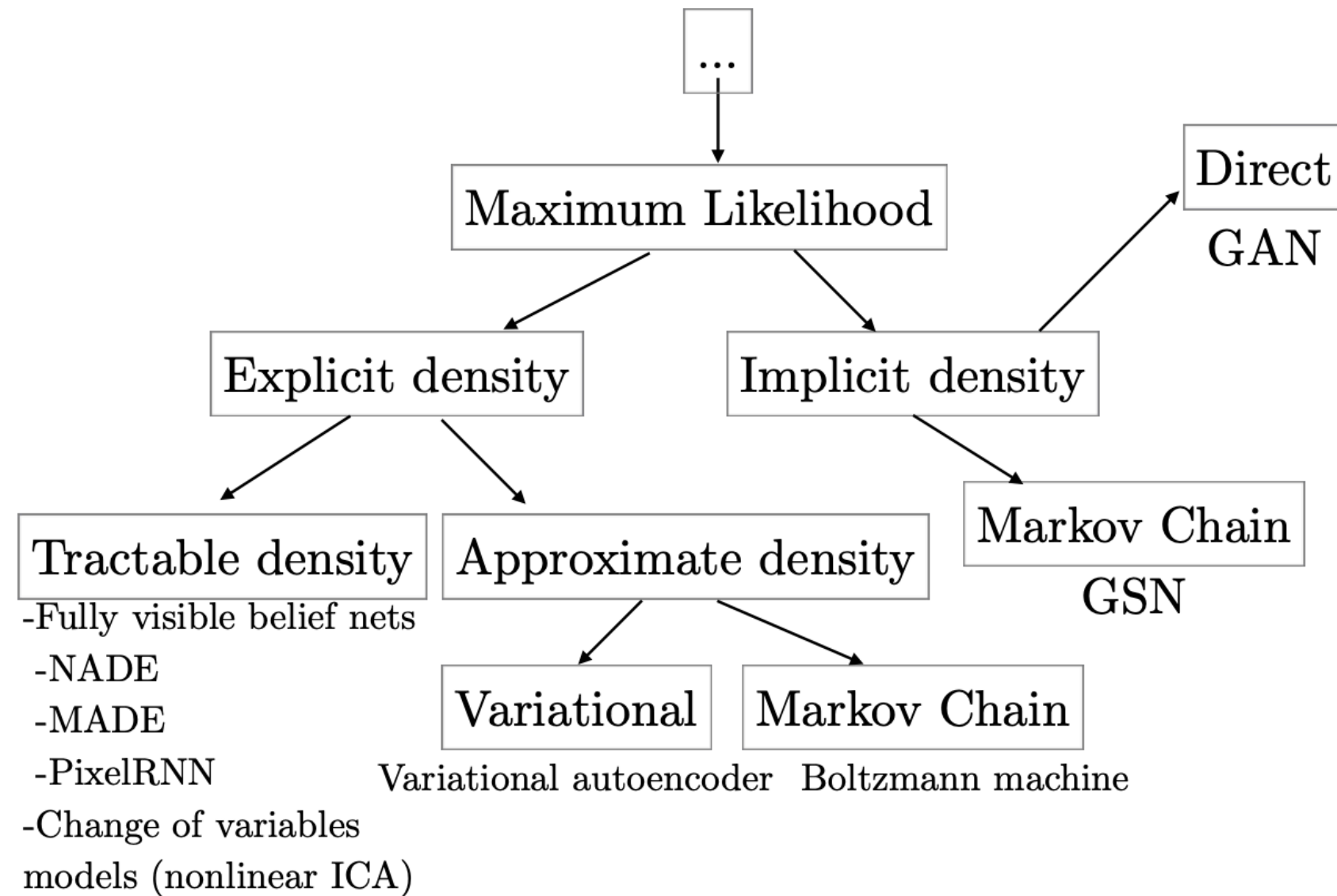
Training objective:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x_i|z)] - D_{\text{KL}}(q_\phi(z|x) || p(z))$$

Intuition: Intuition: reconstruct  $x$  well (first term) while keeping the approximate posterior close to a simple prior (second term).

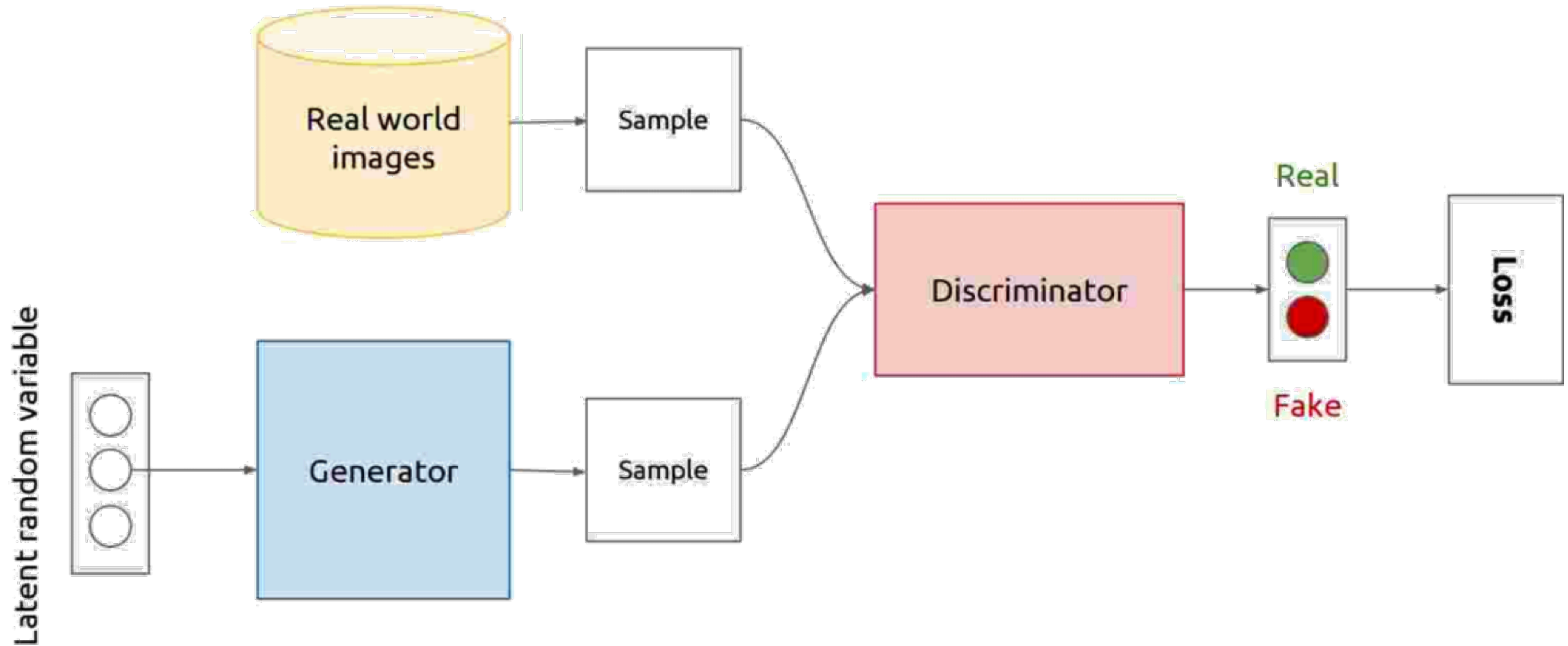
**Question:** Why is the variational autoencoder a generative model, but other autoencoder variants, like the denoising autoencoder, are not?

# Back to the Birds-Eye perspective



# Generative Adversarial Networks

# Key idea of Generative Adversarial Networks (GANs)



## The GAN Training Algorithm

1. Sample real data points  $x$
2. Sample noise  $z$
3. Apply generator  $G_\theta$  to transform noise  $G_\theta(z)$
4. Apply discriminator  $D_\phi$  to distinguish between real  $x$  and fake  $G_\theta(z)$ .
5. Optimize a min max game.

$$\min_{\theta} \max_{\phi} (G_\theta, D_\phi) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D_\phi(X)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D_\phi(G_\theta(z)))]$$

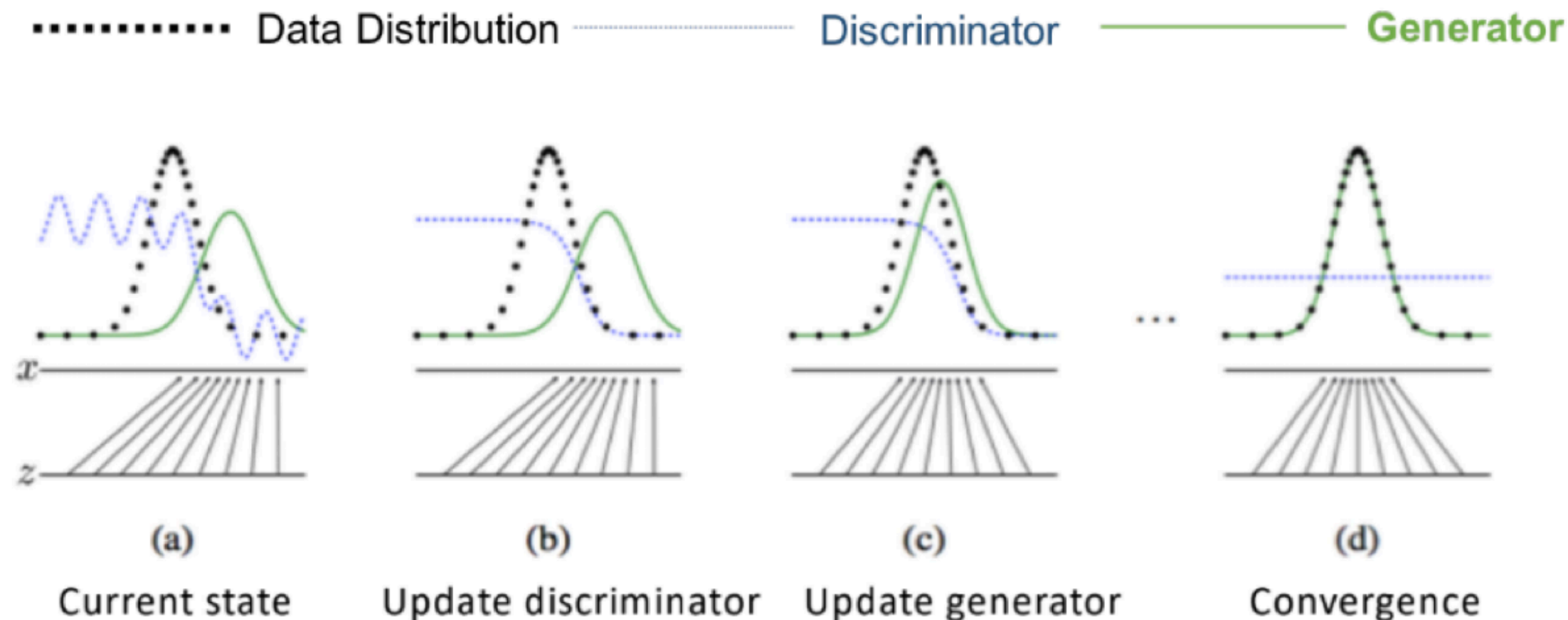
# Which is generated by a GAN?





# Caveats: Min-max optimization is a little unstable

For example: If discriminator becomes too good early on, there is no way the generator can catch up.

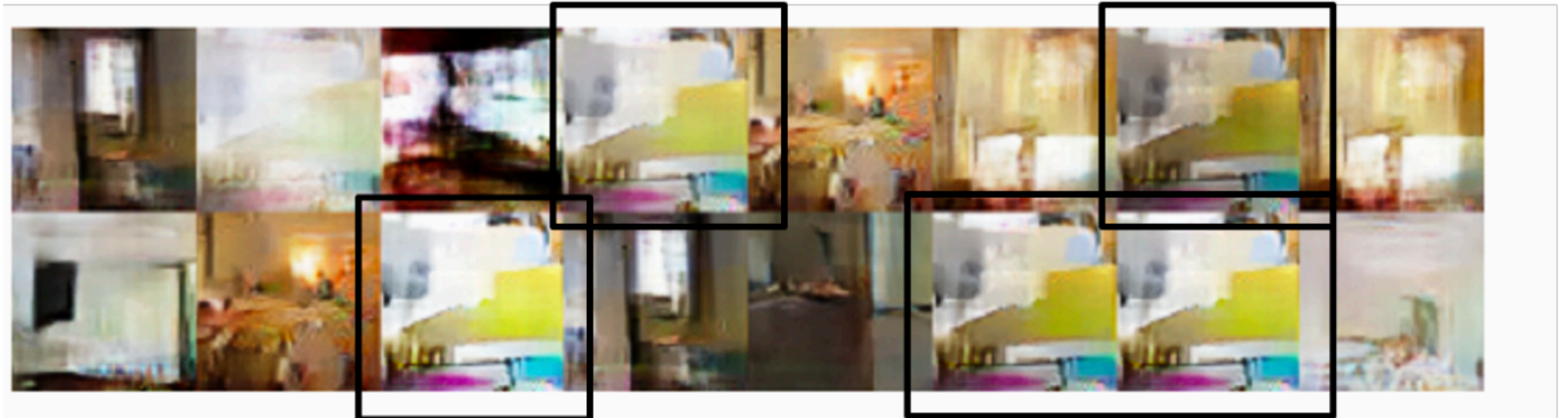


Goodfellow et al., 2014



## Caveat 2: Sometimes there is mode collapse

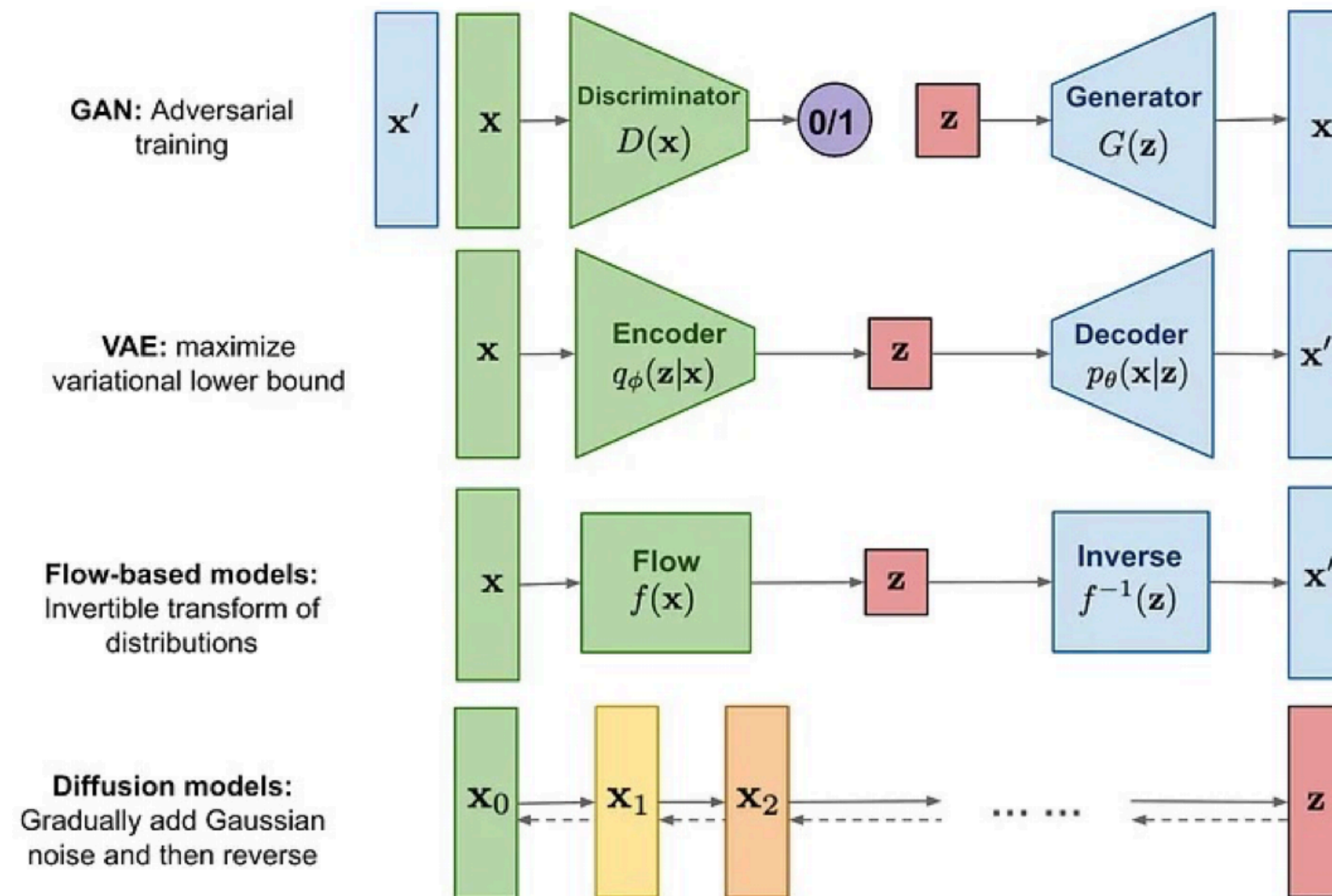
When the generator finds a strategy where it can fool the discriminator well, it may focus too much on this type of images.



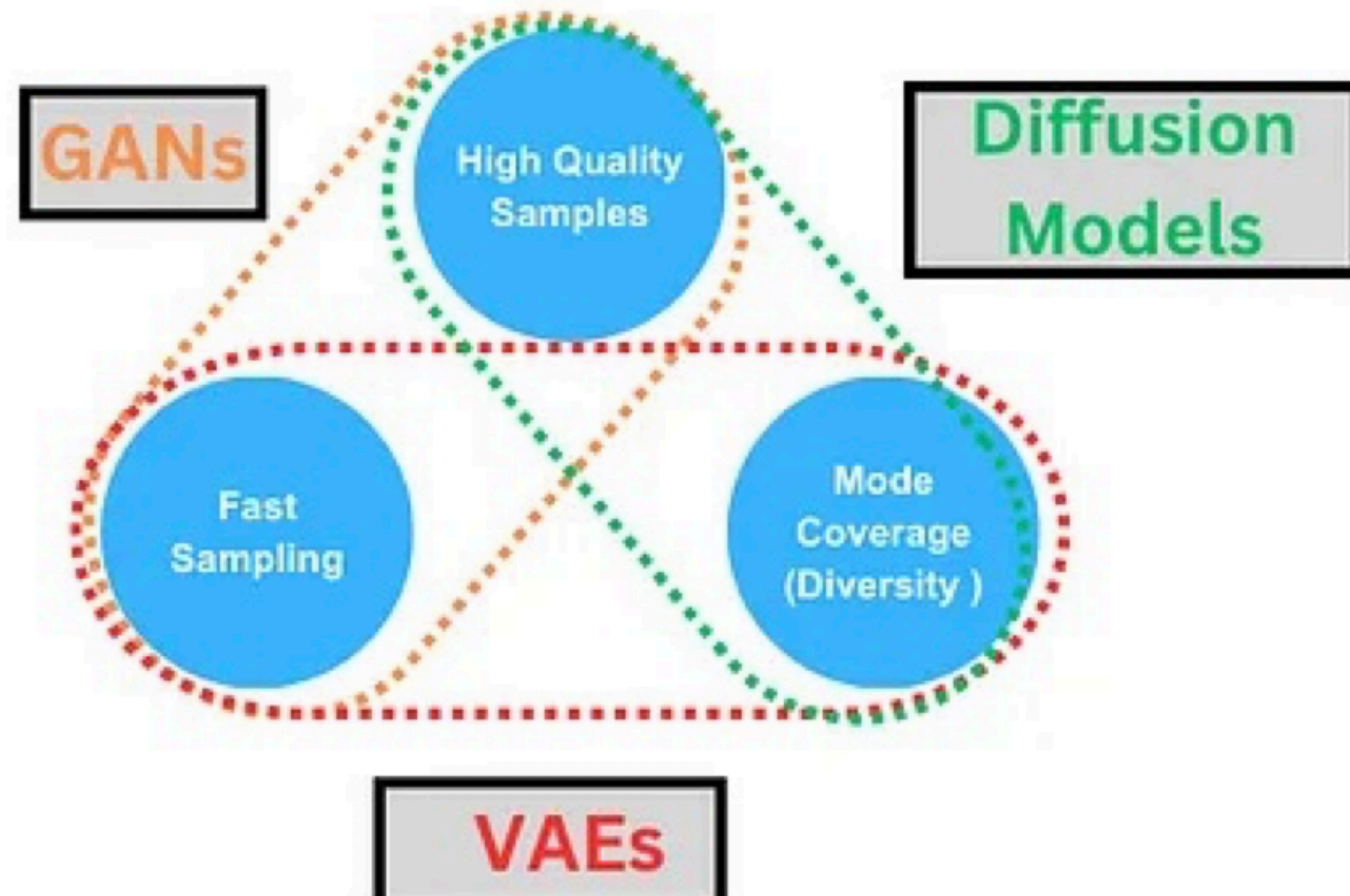
Arjovsky et al., 2017

# Current Image generators are based on Diffusion models

More about that in the Computer Vision course  **SOON** Fall



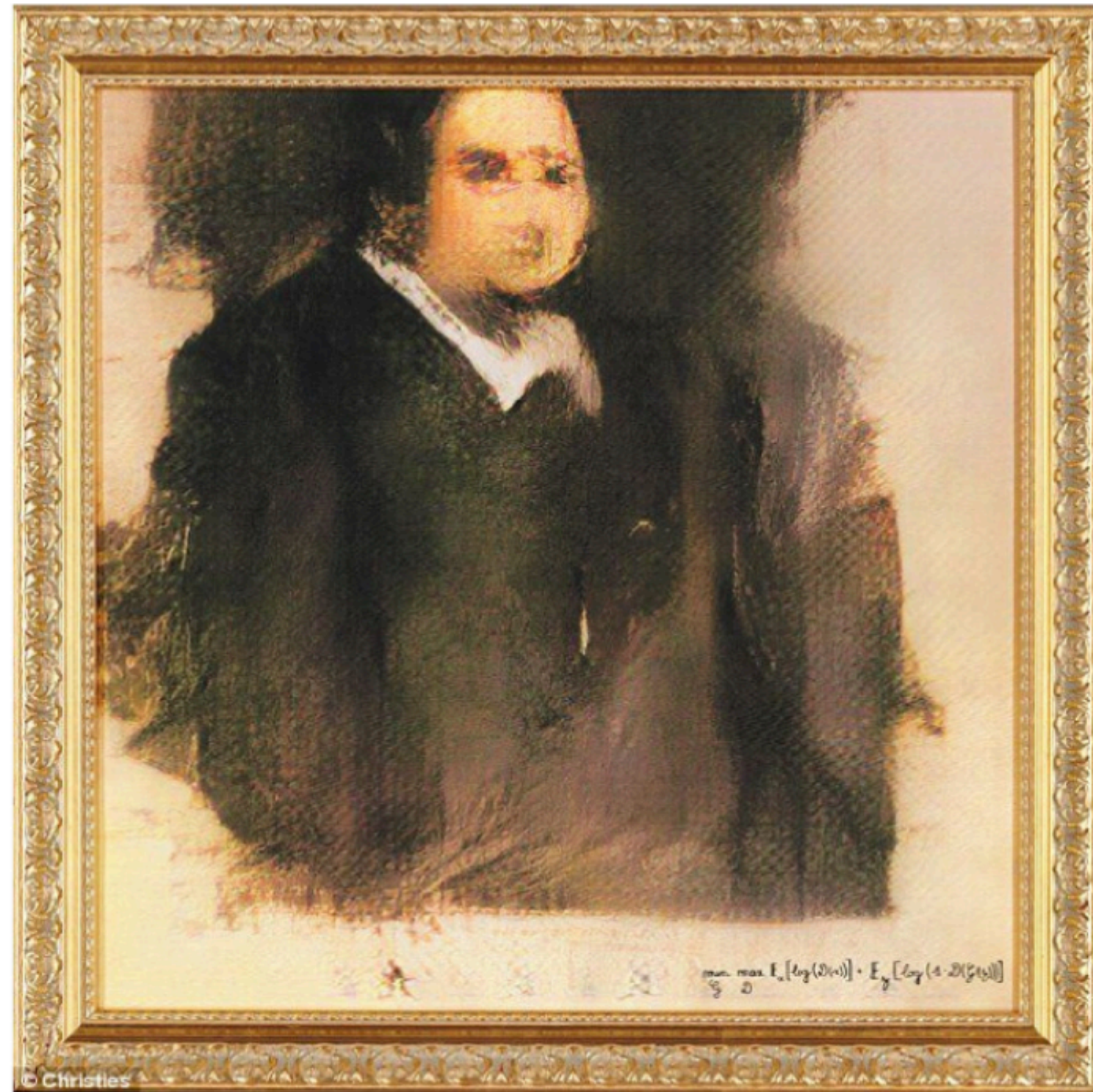
# You cannot have it all ;)



# Keywords

- Energy-based models
- Probabilistic graphical models
- Bayesian Networks vs. Markov Networks
- Neural networks as generative models
- Generative adversarial networks





# How's my teaching?

